

variable-width pulse that can be varied from 1 ms to 2 ms. The width of the pulse controls the position of the servo motor.

A 1ms pulse rotates the shaft to extreme counter-clockwise position (-45°). A 1.5ms pulse places the shaft in a neutral midpoint position (0°). A 2ms pulse rotates the shaft to extreme clockwise position (+45°). The pulse width is sent to the servo motor approximately 50 times a second (50 Hz), which means that the time gap between consecutive pulse widths is approximately 18 ms.

Here the 16-bit timer/counter1 module is configured in the phase- and frequency-correct PWM mode to generate a pulse train of suitable pulse width through output-compare channel B. The output-compare mode for channel B is configured to operate in the inverting mode. The clock source for the timer is the I/O clock cycle prescaled by a factor of 8.

When connecting the servo motor, make sure that the position-control lead for the servo motor is connected to PD4 pin (pin 18) of the microcontroller and the power leads to the power rails available on the development board.

The different modes of PWM operation available in ATmega32 are described below.

Normal mode. It is the simplest mode of operation (WGM13:0=0). In this mode, the counting direction is always up (incrementing), and no counter clear is performed.

CTC mode. In CTC mode, WGM13:0 = 4 or 12, and OCR1A or ICR1 register is used to manipulate the counter resolution. The counter is cleared to zero when the count value (TCNT1) matches either OCR1A (WGM13:0 = 4) or ICR1 (WGM13:0 = 12).

Fast PWM mode. The fast PWM mode (WGM13:0 = 5, 6, 7, 14 or 15) provides a high-frequency PWM waveform generation option. It differs from other PWM options in its single-slope operation. The counter counts from

minimum to maximum value, then restarts from minimum.

The PWM resolution for fast PWM can be fixed to 8-, 9- or 10-bit, or defined by either ICR1 or OCR1A. The minimum resolution allowed is 2-bit (ICR1 or OCR1A set to 0x0003) and the maximum resolution is 16-bit (ICR1 or OCR1A set to MAX).

Phase-correct PWM mode. The phase-correct PWM mode (WGM13:0 = 1, 2, 3, 10 or 11) provides a high-resolution phase-correct PWM waveform generation option. It is based on a dual-slope operation. The counter counts repeatedly from minimum (0x0000) to maximum (0xFFFF) and then from maximum to minimum.

The PWM resolution for the phase-correct PWM mode can be fixed to 8-, 9- or 10-bit, or defined by either ICR1 or OCR1A. The minimum resolution allowed is 2-bit (ICR1 or OCR1A set to 0x0003) and the maximum resolution is 16-bit (ICR1 or OCR1A set to MAX).

In phase-correct PWM mode, the counter is incremented until the counter value matches either of the fixed values 0x00FF, 0x01FF or 0x03FF (WGM13:0=1, 2 or 3), the value in ICR1 (WGM13:0=10) or the value in OCR1A (WGM13:0=11). The compare units allow generation of PWM waveforms on the OC1x pins. Setting the COM1x1:0 bits to '2' produces a non-inverted PWM and an inverted PWM output can be generated by setting the COM1x1:0 to '3'.

It is recommended to use the phase- and frequency-correct mode instead of the phase-correct mode when changing the maximum value while the timer/counter is running. When using a static maximum value, there is practically no difference between the two modes of operation.

Phase- and frequency-correct PWM mode. The phase- and frequency-correct PWM mode (WGM13:0=8 or 9) provides a high-resolution phase- and frequency-correct PWM waveform generation option. Like the phase-correct PWM mode, it is based on dual-slope operation. The counter

counts repeatedly from minimum to maximum and then from maximum to minimum.

The main difference between the phase-correct and the frequency-correct PWM modes is the time of OCR1x register update.

By now we have discussed almost all the peripheral modules present in the ATmega32 microcontroller, except for those which the microcontroller uses for communication with the outside world. These include modules like the USART (universal synchronous and asynchronous receiver transmitter) module, the I²C module (two-wire interface) and the SPI module (not discussed here) that the controller uses to communicate with other controllers.

Example 11: USART

The USART in ATmega32 is a highly flexible serial communication device. You may go through its details in the datasheet of ATmega32.

The USART has to be initialised before any communication can take place. The initialisation process normally consists of setting the baud rate, setting the frame format and enabling the transmitter or the receiver depending on the usage. For interrupt-driven USART operation, the global interrupt flag should be cleared (and interrupts globally should be disabled) when doing the initialisation.

The uart.c sample source code is used to test the USART module. It prepares a user interface on the HyperTerminal client and interacts with the user seeking the inputs to operate the LEDs, LCD module and buzzer on-board. It is possible to control all these devices on the development board by giving suitable commands on the HyperTerminal window. Asynchronous serial transmission takes place between the PC and the microcontroller with the help of the USART module. In the source code, the USART module is initialised to operate in the asynchronous mode with a frame format of 8-bit data, no parity bit and one stop bit. The baud rate is fixed at 57,600 bps.